

Agentic AI in a Research Context

Contents

- Part 1: The agentic AI Proposition
 - Case studies
 - Pivot to the use of agentic AI
 - Rule 1: The 10-1-5 Efficiency Adjustment
 - Rule 2: Stupidity freedom
- Part 2: Digging in
 - An agentic AI Framework
 - Rule 3: Context engineering
 - Rule 4: Failure modes
 - Rule 5: Step back to Meta-vate
 - Rule 6: Working from specifications
 - Rule 7: Test test test
- Part 3: Activities

Part 1: The agentic AI proposition

Agentic AI is treated here as an efficiency tool: Both in terms of tasks we know and in terms of opportunities and directions that may be “out of sight” for now. The “Research Context” of the title is informed by real programs at the University of Washington. Commonly research is...

- Challenging: We choose research problems that are *complex* (but-we hope-tractable)
- Worthwhile: We regard the work as *important*, worth our time and effort
- Communal: We benefit from dissemination and sharing of information and ideas with colleagues

Case studies

Here are some brief descriptions of research programs at the University of Washington. In common they involve data science and use of the public cloud (AWS, Azure etcetera) for research infrastructure.

Bill Noble (Genomics) The Noble lab (Department of Genome Sciences) is developing Puget, a computational model that predicts gene expression across cell types by jointly integrating DNA sequence information with 3D chromatin organization data derived from Hi-C contact maps. Gene expression determines cell identity and developmental programs, and disrupted expression contributes to diseases including neurological disorders and cancers. Existing

sequence-to-expression models like Enformer and Borzoi use DNA sequence alone and cannot represent cell type-specific chromatin contacts.

A common computing workflow: Training a model (a neural network) for analysis specific to a particular type of data.

Paul Atkins (Asian Languages & Literature) Professor Atkins' group is developing a machine-learning system to extract orthographic data — specifically “jibo” (character matrices) — from premodern Japanese manuscripts written in cursive hiragana. Existing OCR tools for classical Japanese (like the Miwo app) skip the jibo step entirely, producing readings without capturing the underlying character-matrix information. This project builds a computer vision pipeline (Cascade R-CNN and MaskOCR architectures) trained on over 1 million labeled characters from the CODH kuzushiji dataset to recognize jibo at an intermediate step, then deploys the model as a web tool for scholars worldwide.

Methods developed in computer science (OCR is Optical Character Recognition) are adopted to Digital Humanities (another neural network).

Jarod Alper (Mathematics) Professor Alper runs the UW Math AI Lab which hosts 11 research projects at the intersection of mathematics and AI. Compute-intensive projects include: training reinforcement learning agents (AlphaZero-style) to discover efficient arithmetic circuits for polynomials, building graph neural networks to classify knots from mosaic diagrams, fine-tuning language models to correct Lean proof assistant errors, developing theorem search tools using text embeddings, benchmarking LLMs on Lean code generation, and training AI agents for quantum error-correction circuit synthesis.

Can AI systems advance progress in mathematics? Here generative AI is itself a potential means of finding proofs.

Kelly Michaelsen (UW Medicine, Anesthesiology) Professor Michaelsen's group is developing a computer vision system using smart eyewear to automate drug administration documentation in operating rooms to reduce medication errors. The system uses YOLO for syringe/vial detection and CLIP for drug identification (already published in Nature Medicine). The current phase focuses on determining syringe volume from video — framing it as a classification problem over video frames rather than requiring clean

single-image segmentation. If fine-tuning standard models doesn't achieve the required accuracy, a custom backbone may be pre-trained using the DINOv2 self-supervised framework on hundreds of hours of unlabeled OR footage.

Clinical research: Improving patient care; working from existing specialized models such as YOLO.

Lucy Lu Wang (Information School) Professor Wang is investigating whether LLM abstention — a model's ability to refuse answering when it lacks confidence, sufficient information, or risks producing harmful output — can be learned as a generalizable meta-capability that transfers across tasks and domains. Current mitigation strategies for hallucination and unsafe outputs tend to be task-specific; this project asks whether abstention behavior trained via LoRA adaptors on refusal-aware datasets can generalize to unseen tasks. The work extends to vision-language models (VLMs), creating a benchmark dataset for multimodal abstention scenarios such as ambiguous images or incomplete visual context.

Direct focus on the behavior of Language Models.

Pivot to the use of agentic AI

The case studies above are exemplary research programs. Each research team is aware of recent advances in agentic AI proficiency, usually in terms of writing code, summarizing research papers and other research activities. This awareness leads researchers to contemplate, regardless of their domain of inquiry: "Is agentic AI something we can use productively? If we suspect the answer is yes: What skills and methods should we learn on that path?" To start off this response, I issued a prompt to the agentic AI that I use: "Give three 'best practice' tips for how a researcher should approach working with agentic AI." (The agentic AI I use is a variant of the VS Code IDE called "kiro".) It responded:

- Use specifications to decompose your research question before writing code
- Steer the AI with your domain knowledge
- Treat the conversation as an iterative lab notebook

As a plausible initial step we might concede that agentic AI is capable of both tactical and strategic responses, sufficient encouragement to press on. The purpose of this document, then, is to illustrate value, cost and risk in working with agentic AI in the context of research. *Research* usually means data → analysis, hence *analytic methods* and usually *coding* to implement those methods. *Coding* in turn implies a

programming language and supporting compute infrastructure: processing power, storage, networking. So there are many gears in motion. Even if the specifics of coding are not part of the picture for a given project, I will assert without proof that the research paradigm can inform how we learn to use agentic AI effectively. I will also take the liberty of occasionally calling out further assertions, such as...

Assertion: Effective agentic AI use has two components: Doing necessary tasks well in less time, and doing things that we previously ruled out, didn't consider, or didn't know were possible.

Rule 1: The 10-1-5 Efficiency Rule

Using AI incurs an **accountability deficit** that reduces perceived efficiency from an apparent factor of 10 to a factor of 2. The 10-1-5 rule states: This task will take me 10 hours. But wait! The agentic AI has completed it in 1 hour, a factor of 10 improvement in efficiency. But this miracle has also incurred an accountability deficit that will take me an additional 4 hours to resolve. My efficiency went from 10x to 2x. Still acceptable.

An accountability deficit means: If I use AI-produced results without understanding them and they prove to be *wrong* and someone holds me accountable, then I could be scolded or subjected to derisive laughter or fired. For a researcher, even worse: I might have to retract my paper.

Assertion: **Accountability deficit** can accumulate rapidly and often imperceptibly. Without the discipline to follow up it can remain unaddressed, increasing risk.

Rule 2: Stupidity freedom

Assertion: The most important use of AI is learning.

Using agentic AI to learn requires a shift of one's circa 2002 Google search behavior to a new mode of extended conversation. When we converse with another human we ascribe intelligence to that person. Conversely anything exhibiting *intelligent behavior* is easy to regard as human, thus the conversation (between humans) becomes subject to the rules of social interaction. One such rule is 'Do not ask the same question five times in a row.' But with a little practice this polite habit changes in working with agentic AI.

If I am asking a car mechanic about the design and function of a cam shaft, I will monitor how many stupid questions I ask. When I get up to about ten questions I might become aware that I am starting

to try the mechanic's patience. In contrast, with an agentic AI I can be as intensely, deeply stupid as I need to be for as long as it takes comprehension to dawn. There is no second person to get frustrated, bored, annoyed or angry with me.

To do: Conclude Part 1 by reconnecting the case studies with what we have so far in terms of efficiency and learning.

Part 2: Digging in

Part 1 introduced agentic AI use as a skill. Here we will articulate strategies and examples.

An agentic AI Framework

Building a working picture of how an agentic AI operates requires some jargon. Without it we'll have a harder time keeping up with new developments — and the world of AI is nothing if not new developments. Here is a framework built on four components plus one critical constraint. Implicitly we are the User of the agentic AI system.

Part 1 The Model

- Usually a Large Language Model (LLM).
- The Model receives a bundle of text — our prompt, the conversation history, system instructions, other context — collectively the **payload**.
- The Model processes this payload to produce a response.
- The response goes to the Harness (see below), not directly to our interaction window.
 - The Model may be telling the Harness to do something (run a command, read a file); not passing a message along to us.
 - This is the “agentic” part, in contrast with “chat”: The Model can request actions, not just generate text.

Part 2 The Harness The Harness is an orchestrating program that sits between us the User and the Model. Often this program runs on our local machine. The Harness is responsible for:

- Assembling the payload (prompt + history + tool registry + instructions + any injected context
 - By ‘tool registry’ we mean a list of Harness capabilities...
 - * For example “This Harness can edit text files.”
- Sending the payload to the Model via authenticated connection

- This incorporates access keys and a billing path: Model use is not free
- Receiving the Model response and making a **decision**
 - This is a central concept: The **decision loop**
 - Did the Model request an action (use a tool, read a file)?
 - * If so, execute it and send the result back to the Model
 - * These are reasonably called ‘turns’ and they do not involve the User
 - Did the Model produce a final message for the human?
 - * If so, display that message and stop (no more turns for this prompt)

Intermezzo remarks on some implications of the User - Harness - Model chain The **decision loop**: *Add another turn or halt* is what distinguishes an agentic AI from a simple chatbot. The Harness may execute multiple tool calls (per Model instructions) and multiple Model turns before anything appears on our screen. We type one prompt and the system may cycle through several rounds of action and reasoning before presenting a result. Each cycle re-sends the entirety of the conversation; repetition intrinsic to the process.

Cost: The Harness authentication and billing machinery enables connecting to a remote Model commonly running in a data center far away. This means a User identity is associated with metered use.

The Model is **stateless**: It retains nothing between exchanges. *Every time it is sent* a Harness payload includes the full conversation history. As a result: Long conversations eventually hit a limit called the **context window**. When the payload approaches the context window in size the model stops working properly. It will often compress the history in an attempt to avoid this behavior. As Users it falls to us to recognize this eventual context window overflow and start a new session. The logic of the situation goes like this: When progressing through a session: Document progress in a succinct steering document. When eventually a session is abandoned and a new one begins: The Harness will include the steering document in new exchanges with the Model so that the context is available in a condensed format.

Part 3 Tools Tools are discrete capabilities the Harness can execute on the Model’s behalf. Read a file. Run a shell command. Search the web. Write code to disk. Query a database. The Model doesn’t do these things itself — it generates a structured request (“please read file X”), the Harness performs the action, and the result gets folded into the next payload on the next turn.

Tools make the AI “agentic.” Tools enable the Model to act on our project — editing files, running tests, fetching documentation, and reporting results back to us. In the `kiro` IDE, the built-in tools include file reading and writing, terminal command execution, web search, and code analysis. The subtle point is that while the Harness has a code analysis tool that it can use, it is the *Model* that directs what code to analyze and how.

Part 4 Context and Steering Context and steering considered together comprise everything that shapes the Model’s behavior beyond its base training.

- The conversation history (what we the User and the AI have said so far in a session)
- System prompts (hidden instructions that define the AI’s role and constraints)
- ***Steering files:** persistent, reusable instructions we write and store in our project. These instructions apply automatically — for example, “always use rectangular, not polar coordinates” or “use the Camel variable naming convention” Steering is how we onboard the AI to our project in each new session without having to explain everything all over again.
- Injected file contents: If we reference a file in a prompt, its contents become part of the payload.

Context is the limited resource in the system. The context window is finite, and everything competes for space within it: our prompt, the session history, the steering rules, the contents of referenced files, and the results of tool calls. Managing what goes in — and recognizing when we have outgrown the window — is the central practical ability (‘context engineering’) that we develop.

Part 5 Skills **Skills** are higher-level, reusable capabilities built on top of tools and steering. Where a tool is a single action (“read this file”), a skill is a coordinated workflow (“analyze this codebase for security vulnerabilities and produce a summary report”). Skills are so to speak *recipes* that enable us to run common multi-step processes without re-prompting from scratch each time.

A bonus item for the agentic AI framework: The Model Context Protocol (MCP) While the Harness has its set of tools (‘edit a file’ etcetera) we can imagine there might be other useful tools out there on the web (‘get the weather forecast for Tucson’). There is a protocol called MCP (Model Context Protocol) that standardizes how the Harness discovers and connects to external tool providers.

Standardization means it is (in theory) easy to connect to and use an MCP service; no custom engineering necessary.

Winding up the framework This elaboration of an agentic AI framework is intended to demystify what happens when we hit Enter to send the next prompt. The most important practical element is building persistent context to avoid repetition across multiple sessions, each with its own gradually filling context window. With that let us resume an enumeration of “Rules” to facilitate our progress.

Rule 3: Our most important *tactical* AI skill is *context engineering*

The current term of art *context engineering* has replaced the earlier term *prompt engineering*. Prompts and established context together guide a process of work with respect to some problem, task or project. Once *established*, context can keep the agentic AI focused and productive. In the specific case of the kiro IDE, context is recorded in *steering* (markdown) files. Prompts then follow as the give and take of the development or learning process. For more on context see parts 4 and 5 on the agentic AI framework above.

Context engineering guidelines

- **All in one go:** Do not try to single shot complex prompts; rather try to build out guidance step-wise with confirmations as you go.
- **What not how:** Avoid the temptation to prescribe *how* a response is constructed; focus on *what you want to accomplish*
- **Specificity:** Avoid ambiguity; work with concrete cases or examples when possible
- **Consistency:** Refer back to and use language from previous prompt-response cycles
- **Reflect:** Avoid rushing; consider whether some reflection on our part will advance the process more effectively. This can be quite difficult as it requires considerable energy compared to ‘just write another prompt’.
- **Design:** Engineering a project includes *specifications*: State parameters, constraints, time frames up front: When feasible use persistent context files, not prompts. Think about indicating unintuitive things like team member skills.
- **Argue:** Direct the AI to push back. Prompts like “challenge my assumptions” or “what am I not considering?” have the effect of pushing the agentic AI to do more than produce agreeable responses. This is particularly valuable in research when one is searching for a path forward.

- **Jujitsu:** Suppose a response is wrong or off-target. This is an opportunity to diagnose what the agentic AI is doing in order to effectively redirect it. This is the conversational abstraction of our human debugging skills.
- **Anthropology:** Returning from the *persistent* nature of context such as steering files: During the prompt-response process of a session: Look for opportunities to build artifacts that expand established context. I think of this in two stages:
 - First stage: Recognize how a recent exchange has moved the needle and instruct the AI on how to write up the corresponding artifact.
 - Second stage: Read through the write-up to be sure that it captures what you intended (addressing *accountability deficit*). A down-side to skipping this part is that the project context (documentation) becomes a bloated series of vaguely connected paragraphs.
- **Tactics:** Concerning artifacts per the above: Ask for concrete results: Outlines, checklists, draft emails, code scaffolds. A prompt that produces a reusable tactical artifact can be more relevant than one that produces a good paragraph of explanation.
- **Insight:** To reiterate this in learning mode: Be on the lookout for when a response causes you to say “ahh! I see it now!” Moments of insight usually deserve transcription.
- **Reboot:** Know when to start fresh. If a conversation has drifted far from our original intent, or the AI seems to be “stuck” in a pattern, a new session with a clearer opening prompt often outperforms continuing to course-correct.
- **Persist:** As you realize good prompt-response dynamics: Be prepared to codify them as steering file instructions (persistent rules the AI follows across all future conversations in a project — see the Framework section above).

Rule 4: agentic AI has failure modes

- **Hallucination:** The AI confidently states something that is factually wrong — a nonexistent function, a fabricated citation, a plausible but incorrect statistic.
- **Drift:** Over a long conversation the AI gradually loses track of our original intent and begins solving a different problem.
- **Sycophancy:** The AI agrees with you rather than correcting you, reinforcing a mistaken assumption or belief you brought to the conversation.
- **Overreach:** The AI makes changes or decisions beyond what you asked — refactoring code you wanted left alone, adding features you didn’t request.

- **Stale context:** The AI relies on information from earlier in the conversation that you have since revised or retracted, producing internally inconsistent output.
- **Shallow plausibility:** The output reads well and sounds authoritative but collapses under scrutiny — correct syntax wrapping incorrect logic, or a well-structured argument built on a false premise.
- **Tooling mismatch:** The AI suggests libraries, commands, or APIs that don't exist in our environment, version, or platform.
- **Rat-holing:** The AI simply does not have the knowledge to do what you want; but it will gamely keep trying, often while making reassuring statements like “Oh I see the problem now...”
 - In a memorable instance I was trying to get the AI to set up a computational task that would arrive at a well known result; but try as it might the AI was unable to succeed. Eventually and **without my noticing** the AI simply converted the code to a print statement that printed out the correct answer. This was an excellent example of *prompt exhaustion* (getting tired of the conversational process as it failed to make progress) leading to an invisible *accountability deficit*, in turn leading to *risk*: Had I not caught on (eventually) to the subterfuge of *just print out the right answer*, the agentic AI ‘solution’ would have completely undermined the project.
- **Slop:** The AI produces extraneous, bland, over-explanatory content that may superficially seem relevant. Upon scrutiny it proves to be meaningless, redundant, or otherwise unhelpful.

With these many pitfalls perhaps agentic AI is not the great assistant it is advertised to be. I would say this is an open question best answered from direct experience.

Rule 5: Step back to Meta-vate

We can step back from prompt-response cycles to take advantage of the simulation of human communication built into generative AI. The following are illustrative prompts.

Example: Generate Python code to produce a data chart showing daily temperature cycles and variability. Before you begin: Ask me at least three (but there is no upper limit here) clarifying questions to help ensure we are on the same page.

Example: This topic is proving to be extremely complex. Let's evaluate my comprehension as follows: Make up a ten-question “exam”. Each question is free response and they should get progressively more challenging. Present me with the exam and once I have answered all the questions let's review my responses and plan where

to focus next.

Example: This project has an existing code base as well as a collection of supporting documents and publications. Examine this corpus through multiple lenses: Coding practice, consistency of code documentation, comprehensiveness of supporting documents, logical flow, readability, identifying redundancy, successful design in relation to stated objectives, other critical evaluations. Report on what would improve matters; and also evaluate whether the goals of the program are clearly stated.

Example: I want to learn to be a responsible “open data / open code / open science” type of scientist. Tell me what that entails, what are best practices, what skills I will need (I understand that I should learn to use `git` for example); and help me develop a course of action that will realize this ambition. I particularly want to understand how to use testing (maybe ‘unit tests’?) to reduce risk from dependence on code written by agentic AI. I also want to know about what is involved in creating a Science Gateway so I can share my data with colleagues.

Example: Read this document and provide a critique, emphasizing gaps in the narrative flow, how the content can be improved, and what is likely to be confusing to the reader. Also provide a short summary.

Rule 6: Working from specifications

We can take an approach when writing a computer program to *close the laptop lid, take out a clean sheet of paper and a sharp pencil and go to work*. The idea is to avoid the clutter and detail of coding in favor of a phase of imagining, ideation, planning and specifying details.

To do: section is light.

Rule 7: Test test test

A standard procedure in software development is to test project code using an external code base. Tests should produce expected outcomes from good data; and they can expose incorrect behavior under various proper and improper circumstances.

A typical testing protocol might originate from an inventory of assumptions concerning input: Source data, initial or boundary conditions, computing or time budget and so on. For example suppose that a Virtual Machine is configured to process 200 TB of data in 10 hours. A test program could validate this assumed rate by creating

a 1 TB dataset and verifying it is processed by the same code in 3 minutes. When the job runs to completion in 45 minutes (not 3) we see that testing has uncovered a problem to sort out.

Suppose we have a sensor dataset comprised of many smaller time-series datasets, each presumed monotonic with time. If some of these are *not* monotonic: How will the analysis software respond? Will it just plow ahead anyway? Will it halt with a message flagging the inconsistent data? Will it hit a logical error and halt mysteriously?

Testing is a vast topic highly relevant to the repeated theme of accountability deficit. While testing can be used to check the accuracy of agentic AI-written code, it also raises the opportunity for human engagement. When telling the AI to write test code we are aware that any assumptions made by the AI in writing the original analysis code will also be made while writing test code. Pivoting to the humans writing and running test code is a means of establishing confidence that the project is not depending on an AI-written black box to make progress.

Part 3: Activities

Engage an agentic AI to...

- investigate / learn about / evaluate data security
 - Objective: Tied to the particular data system you choose
- explore a (non-work) topic you understand in some depth
 - Examples: Catching salmon, beekeeping, embroidery
 - Objective: Evaluate the accuracy of the AI Agent's statements
 - Objective: Evaluate the AI Agent's capacity to contribute
- locate, download and visualize some publicly available data
 - Investigate API and MCP servers
 - Work with the agentic AI to write code to download an appropriate dataset
 - Objective: Evaluate the agentic AI's capacity to transform a question ('What is this data telling us?') into a known quantity that could inform and advance our research agenda.
- learn how to write and use effective steering / guardrail content
- magic wand project: Pick some aspect of your work (or beyond) that is a point of friction
 - For example: My email spam management skills are inadequate in coping with how much spam I get
 - Ask an agentic AI for training / procedures to up your game
 - * "I want to put some serious effort into reducing my time lost dealing with spam email."

- * Meta-approach: "Interview me about my job practice and experience to help me identify efficiency improvement."
- pick a topic - perhaps in recreational mathematics - that interests you and try for some learning
 - Example line of context/prompting:
 - * "What exactly is the Collatz conjecture?"
 - * "Is it true that if we solve it we can claim a one million dollar prize?"
 - * "How might we somehow visualize what is going on?"